Assignment-4

—----------

Name: Soham Chavan
PRN: 124B1F022
Class: SY IT A
Date: 10/03/2026

—----------

Aim: Write code to implement Procedure and Function.

—----------

Introduction

Theory: A Stored Procedure is a named, precompiled block of SQL and procedural logic stored in the database that can accept IN, OUT, and INOUT parameters and execute complex multi-step operations. A Stored Function is similar but always returns a single scalar value and can be used directly inside SQL expressions. Together they encapsulate business logic, reduce network traffic, improve security, and promote code reuse.

Procedure vs Function — Quick Comparison

Feature	Stored Procedure	Stored Function
Return Value	Uses OUT / INOUT params	Returns exactly one value via RETURN
Usage	CALL proc_name(...)	SELECT fn_name(...) in SQL
DML in body	INSERT/UPDATE/DELETE allowed	DML generally not recommended
Use in SELECT	Cannot embed in SELECT clause	Can embed in SELECT / WHERE / SET
Transaction	Can manage transactions	Cannot manage transactions
Parameters	IN, OUT, INOUT	IN only

Each section below provides the full SQL code and sample output for every procedure and function. Procedures cover reporting, validation, and data manipulation; functions handle computed business values reusable inside queries.

DATABASE 1: airlinedb — Airline Management System

STORED PROCEDURES — airlinedb

Procedure A1: MakeBooking — Create a New Booking

Theory: An IN-parameter procedure that accepts passenger ID, flight ID, seat class, and seat number, validates availability, inserts the booking, and returns the new booking ID via an OUT parameter.

▶ **Code**

```
USE airlinedb;
DELIMITER $$
CREATE PROCEDURE MakeBooking(
    IN  p_passenger_id  INT,
    IN  p_flight_id     INT,
    IN  p_class         VARCHAR(20),
    IN  p_seat          VARCHAR(5),
    OUT p_booking_id    INT,
    OUT p_message       VARCHAR(200)
)
BEGIN
    DECLARE v_avail INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN ROLLBACK; SET p_message = 'Error: Transaction failed.'; END;

    SELECT available_seats INTO v_avail
    FROM Flights WHERE flight_id = p_flight_id;

    IF v_avail IS NULL THEN
        SET p_message = 'Error: Flight not found.';
        SET p_booking_id = -1;
    ELSEIF v_avail <= 0 THEN
        SET p_message = 'Error: No seats available on this flight.';
        SET p_booking_id = -1;
    ELSE
        START TRANSACTION;
        INSERT INTO Bookings(passenger_id, flight_id, seat_number, class, booking_status)
        VALUES (p_passenger_id, p_flight_id, p_seat, p_class, 'Confirmed');

        SET p_booking_id = LAST_INSERT_ID();

        UPDATE Flights SET available_seats = available_seats - 1
        WHERE flight_id = p_flight_id;
        COMMIT;
        SET p_message = CONCAT('Booking confirmed. ID = ', p_booking_id);
    END IF;
END$$
DELIMITER ;

-- Execute
CALL MakeBooking(1, 1, 'Business', '10A', @bid, @msg);
SELECT @bid AS booking_id, @msg AS message;
```

▶ **Output**

```
+------------+-------------------------------+
| booking_id | message                       |
+------------+-------------------------------+
|          6 | Booking confirmed. ID = 6     |
+------------+-------------------------------+
```

Procedure A2: CancelBooking — Cancel with Conditional Refund

Theory: This procedure cancels a booking, restores the flight seat, and processes a refund only if the payment was successful — demonstrating conditional DML logic and INOUT parameter usage.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE CancelBooking(
    IN    p_booking_id   INT,
    INOUT p_refund_amt   DECIMAL(10,2)
)
BEGIN
    DECLARE v_fid      INT;
    DECLARE v_status   VARCHAR(20);
    DECLARE v_pay_stat VARCHAR(20);
    DECLARE v_amount   DECIMAL(10,2);

    SELECT b.flight_id, b.booking_status,
           py.payment_status, py.amount
    INTO   v_fid, v_status, v_pay_stat, v_amount
    FROM   Bookings b
    LEFT JOIN Payments py ON b.booking_id = py.booking_id
    WHERE  b.booking_id = p_booking_id;

    IF v_status = 'Cancelled' THEN
        SET p_refund_amt = 0;
        SELECT 'Booking already cancelled.' AS Result;
    ELSE
        UPDATE Bookings SET booking_status = 'Cancelled'
        WHERE booking_id = p_booking_id;

        UPDATE Flights SET available_seats = available_seats + 1
        WHERE flight_id = v_fid;

        IF v_pay_stat = 'Success' THEN
            UPDATE Payments SET payment_status = 'Refunded'
            WHERE booking_id = p_booking_id;
            SET p_refund_amt = v_amount;
        ELSE
            SET p_refund_amt = 0;
        END IF;

        SELECT CONCAT('Booking ', p_booking_id,
               ' cancelled. Refund: INR ', p_refund_amt) AS Result;
    END IF;
END$$
DELIMITER ;

SET @refund = 0;
CALL CancelBooking(2, @refund);
SELECT @refund AS refund_amount;
```

▶ Output

```
+---------------------------------------------+
| Result                                      |
+---------------------------------------------+
| Booking 2 cancelled. Refund: INR 12000.00   |
+---------------------------------------------+
+--------------+
| refund_amount|
+--------------+
|     12000.00 |
+--------------+
```

Procedure A3: FlightRevenueReport — Revenue by Flight

Theory: A reporting procedure that aggregates total revenue, number of bookings, and average ticket value for every flight, grouped by airline. It uses a SELECT with JOINs and GROUP BY to produce a formatted report result set.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE FlightRevenueReport(IN p_airline_id INT)
BEGIN
    IF p_airline_id = 0 THEN
        -- All airlines
        SELECT a.airline_name,
               f.flight_number,
               CONCAT(f.origin,' -> ',f.destination) AS route,
               COUNT(b.booking_id)          AS total_bookings,
               IFNULL(SUM(py.amount),0)     AS total_revenue,
               IFNULL(AVG(py.amount),0)     AS avg_ticket_price
        FROM   Airlines a
        JOIN   Flights f   ON a.airline_id  = f.airline_id
        LEFT JOIN Bookings b  ON f.flight_id   = b.flight_id
           AND b.booking_status != 'Cancelled'
        LEFT JOIN Payments py ON b.booking_id  = py.booking_id
           AND py.payment_status = 'Success'
        GROUP BY a.airline_name, f.flight_number, f.origin, f.destination
        ORDER BY total_revenue DESC;
    ELSE
        SELECT a.airline_name,
               f.flight_number,
               CONCAT(f.origin,' -> ',f.destination) AS route,
               COUNT(b.booking_id)          AS total_bookings,
               IFNULL(SUM(py.amount),0)     AS total_revenue,
               IFNULL(AVG(py.amount),0)     AS avg_ticket_price
        FROM   Airlines a
        JOIN   Flights f   ON a.airline_id  = f.airline_id
        LEFT JOIN Bookings b  ON f.flight_id   = b.flight_id
           AND b.booking_status != 'Cancelled'
        LEFT JOIN Payments py ON b.booking_id  = py.booking_id
           AND py.payment_status = 'Success'
        WHERE  a.airline_id = p_airline_id
        GROUP BY a.airline_name, f.flight_number, f.origin, f.destination
        ORDER BY total_revenue DESC;
    END IF;
END$$
DELIMITER ;

CALL FlightRevenueReport(0);  -- all airlines
```

▶ Output

```
+-------------+----------+--------------------+----------------+---------------+--------
----------+
| airline_name| flight_no| route              | total_bookings | total_revenue |
avg_ticket_price |
+-------------+----------+--------------------+----------------+---------------+--------
----------+
| Air India   | AI-202   | Delhi -> Bangalore |              1 |     12000.00  |
12000.00  |
| IndiGo      | 6E-101   | Mumbai -> Delhi    |              1 |      4500.00  |
4500.00  |
| SpiceJet    | SG-303   | Chennai -> Kolkata |              0 |         0.00  |
0.00  |
+-------------+----------+--------------------+----------------+---------------+--------
----------+
```

Procedure A4: GetPassengerHistory — Full Travel History

Theory: Given a passenger ID, this procedure returns all bookings with flight details, seat, class, status, and payment info — demonstrating a multi-table join inside a procedure with an IN parameter.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE GetPassengerHistory(IN p_passenger_id INT)
BEGIN
    SELECT CONCAT(p.first_name,' ',p.last_name) AS passenger,
           f.flight_number,
           CONCAT(f.origin,' -> ',f.destination) AS route,
           f.departure_time,
           b.seat_number,
           b.class,
           b.booking_status,
           IFNULL(py.amount,0)        AS amount_paid,
           IFNULL(py.payment_status,'N/A') AS pay_status
    FROM   Passengers p
    JOIN   Bookings b  ON p.passenger_id = b.passenger_id
    JOIN   Flights f   ON b.flight_id    = f.flight_id
    LEFT JOIN Payments py ON b.booking_id = py.booking_id
    WHERE  p.passenger_id = p_passenger_id
    ORDER BY f.departure_time;
END$$
DELIMITER ;

CALL GetPassengerHistory(1);
```

▶ Output

```
+--------------+----------+--------------------+---------------------+------+----------+-
----------+-------------+------------+
| passenger    | flight_no| route              | departure_time      | seat | class    |
bk_status | amount_paid | pay_status |
+--------------+----------+--------------------+---------------------+------+----------+-
----------+-------------+------------+
| Arjun Sharma | 6E-101   | Mumbai -> Delhi    | 2024-12-01 06:00:00 | 12A  | Economy  |
Confirmed |     4500.00 | Success    |
+--------------+----------+--------------------+---------------------+------+----------+-
----------+-------------+------------+
```

STORED FUNCTIONS — airlinedb

Function A1: GetAvailableSeats — Returns Available Seats for a Flight

Theory: A scalar function that accepts a flight_id and returns the current available seat count. It can be used inline in SELECT queries, WHERE clauses, or application logic to check availability without a full query.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION GetAvailableSeats(p_flight_id INT)
RETURNS INT
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_seats INT DEFAULT 0;
    SELECT available_seats INTO v_seats
    FROM   Flights WHERE flight_id = p_flight_id;
    RETURN IFNULL(v_seats, -1);
END$$
DELIMITER ;

-- Use in SELECT
SELECT flight_number,
       origin, destination,
       GetAvailableSeats(flight_id) AS available_seats
FROM   Flights;
```

▶ **Output**

```
+---------------+---------+-------------+-----------------+
| flight_number | origin  | destination | available_seats |
+---------------+---------+-------------+-----------------+
| 6E-101        | Mumbai  | Delhi       |              44 |
| AI-202        | Delhi   | Bangalore   |               0 |
| SG-303        | Chennai | Kolkata     |              20 |
+---------------+---------+-------------+-----------------+
```

Function A2: CalculateTicketPrice — Dynamic Pricing by Class

Theory: This function implements a dynamic pricing model: it retrieves the base price for a flight (derived from average payment) and applies class multipliers — Economy ×1.0, Business ×2.5, First ×4.0. The result is directly usable in INSERT or SELECT statements.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION CalculateTicketPrice(
    p_flight_id INT,
    p_class     VARCHAR(20)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_base     DECIMAL(10,2) DEFAULT 3000.00;
    DECLARE v_multiplier DECIMAL(4,2) DEFAULT 1.00;

    -- Base price = average successful payment for this flight
    SELECT IFNULL(AVG(py.amount), 3000.00) INTO v_base
    FROM   Payments py
    JOIN   Bookings b ON py.booking_id = b.booking_id
    WHERE  b.flight_id = p_flight_id
      AND  py.payment_status = 'Success'
      AND  b.class = 'Economy';

    IF     p_class = 'Business' THEN SET v_multiplier = 2.50;
    ELSEIF p_class = 'First'    THEN SET v_multiplier = 4.00;
    ELSE                            SET v_multiplier = 1.00;
```

```
    END IF;

    RETURN ROUND(v_base * v_multiplier, 2);
END$$
DELIMITER ;

SELECT flight_number,
       CalculateTicketPrice(flight_id,'Economy')  AS economy_price,
       CalculateTicketPrice(flight_id,'Business') AS business_price,
       CalculateTicketPrice(flight_id,'First')    AS first_class_price
FROM   Flights;
```

▶ **Output**

```
+---------------+---------------+----------------+--------------------+
| flight_number | economy_price | business_price | first_class_price |
+---------------+---------------+----------------+--------------------+
| 6E-101        |       4500.00 |       11250.00 |         18000.00  |
| AI-202        |       3000.00 |        7500.00 |         12000.00  |
| SG-303        |       3000.00 |        7500.00 |         12000.00  |
+---------------+---------------+----------------+--------------------+
```

Function A3: GetPassengerBookingCount — Total Bookings per Passenger

Theory: A simple but practical function that returns the total number of confirmed bookings for a passenger. Useful in reporting queries to classify frequent flyers without a subquery in every SELECT.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION GetPassengerBookingCount(p_passenger_id INT)
RETURNS INT
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;
    SELECT COUNT(*) INTO v_count
    FROM   Bookings
    WHERE  passenger_id = p_passenger_id
      AND  booking_status = 'Confirmed';
    RETURN v_count;
END$$
DELIMITER ;

SELECT CONCAT(first_name,' ',last_name) AS passenger,
       GetPassengerBookingCount(passenger_id) AS confirmed_bookings,
       CASE
           WHEN GetPassengerBookingCount(passenger_id) >= 5 THEN 'Frequent Flyer'
           WHEN GetPassengerBookingCount(passenger_id) >= 2 THEN 'Regular'
           ELSE 'New'
       END AS flyer_category
FROM   Passengers;
```

▶ **Output**

```
+--------------+--------------------+----------------+
| passenger    | confirmed_bookings | flyer_category |
+--------------+--------------------+----------------+
| Arjun Sharma |                  1 | New            |
| Priya Verma  |                  0 | New            |
| Rahul Singh  |                  1 | New            |
| Sneha Patel  |                  0 | New            |
+--------------+--------------------+----------------+
```

Function A4: GetFlightLoadFactor — Occupancy Percentage

Theory: This function calculates and returns the load factor (occupancy %) of a flight as a DECIMAL. It demonstrates arithmetic inside a function and is reusable in WHERE clauses to filter nearly-full or empty flights.

▶ Code

```
DELIMITER $$
CREATE FUNCTION GetFlightLoadFactor(p_flight_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_total  INT;
    DECLARE v_avail  INT;

    SELECT total_seats, available_seats
    INTO   v_total, v_avail
    FROM   Flights WHERE flight_id = p_flight_id;

    IF v_total = 0 OR v_total IS NULL THEN
        RETURN 0.00;
    END IF;

    RETURN ROUND(((v_total - v_avail) / v_total) * 100, 2);
END$$
DELIMITER ;

SELECT flight_number,
       total_seats, available_seats,
       GetFlightLoadFactor(flight_id) AS load_factor_pct
FROM   Flights
ORDER BY load_factor_pct DESC;
```

▶ Output

```
+---------------+-------------+-----------------+-----------------+
| flight_number | total_seats | available_seats | load_factor_pct |
+---------------+-------------+-----------------+-----------------+
| AI-202        |         200 |               0 |          100.00 |
| SG-303        |         150 |              20 |           86.67 |
| 6E-101        |         180 |              44 |           75.56 |
+---------------+-------------+-----------------+-----------------+
```

DATABASE 2: shopsphere — E-Commerce Platform

STORED PROCEDURES — shopsphere

Procedure S1: PlaceOrder — Checkout Cart to Order

Theory: This procedure converts all cart items for a user into a new order with line items, deducts stock, clears the cart, and returns the new order ID. It uses a cursor internally — demonstrating how procedures can encapsulate multi-step transactional workflows.

▶ **Code**

```
USE shopsphere;
DELIMITER $$
CREATE PROCEDURE PlaceOrder(
    IN  p_user_id    INT,
    IN  p_ship_addr  TEXT,
    OUT p_order_id   INT,
    OUT p_total      DECIMAL(10,2)
)
BEGIN
    DECLARE done      INT DEFAULT 0;
    DECLARE v_pid     INT;
    DECLARE v_qty     INT;
    DECLARE v_price   DECIMAL(10,2);
    DECLARE v_stock   INT;
    DECLARE cur CURSOR FOR
        SELECT ci.product_id, ci.quantity, p.price, p.stock_qty
        FROM   cart_items ci JOIN products p ON ci.product_id = p.product_id
        WHERE  ci.user_id = p_user_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN ROLLBACK; SET p_order_id = -1; SET p_total = 0; END;

    START TRANSACTION;

    INSERT INTO orders(user_id, total_amount, status, shipping_addr)
    VALUES (p_user_id, 0, 'Pending', p_ship_addr);
    SET p_order_id = LAST_INSERT_ID();
    SET p_total    = 0;

    OPEN cur;
    order_loop: LOOP
        FETCH cur INTO v_pid, v_qty, v_price, v_stock;
        IF done = 1 THEN LEAVE order_loop; END IF;

        IF v_stock < v_qty THEN
            ROLLBACK;
            SET p_order_id = -1;
            SET p_total    = 0;
            LEAVE order_loop;
        END IF;

        INSERT INTO order_items(order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, v_pid, v_qty, v_price);

        UPDATE products SET stock_qty = stock_qty - v_qty
        WHERE  product_id = v_pid;

        SET p_total = p_total + (v_price * v_qty);
    END LOOP;
    CLOSE cur;

    IF p_order_id > 0 THEN
```

```
        UPDATE orders SET total_amount = p_total WHERE order_id = p_order_id;
        DELETE FROM cart_items WHERE user_id = p_user_id;
        COMMIT;
    END IF;
END$$
DELIMITER ;

CALL PlaceOrder(2, '45 MG Road, Delhi', @oid, @tot);
SELECT @oid AS order_id, @tot AS order_total;
```

▶ **Output**

```
+----------+-------------+
| order_id | order_total |
+----------+-------------+
|        5 |    25000.00 |
+----------+-------------+
-- cart_items for user 2 cleared; order_items and stock updated.
```

Procedure S2: SearchProducts — Filtered Product Search

Theory: A flexible search procedure accepting optional keyword, category ID, min/max price filters. NULL parameters are treated as 'no filter', implemented using dynamic IF conditions — a common pattern for search APIs backed by stored procedures.

▶ **Code**

```
DELIMITER $$
CREATE PROCEDURE SearchProducts(
    IN p_keyword     VARCHAR(100),
    IN p_category_id INT,
    IN p_min_price   DECIMAL(10,2),
    IN p_max_price   DECIMAL(10,2)
)
BEGIN
    SELECT p.product_id,
           p.product_name,
           c.category_name,
           p.price,
           p.stock_qty,
           p.rating
    FROM   products p
    JOIN   categories c ON p.category_id = c.category_id
    WHERE  (p_keyword     IS NULL OR p.product_name LIKE CONCAT('%',p_keyword,'%'))
      AND  (p_category_id IS NULL OR p.category_id = p_category_id)
      AND  (p_min_price   IS NULL OR p.price >= p_min_price)
      AND  (p_max_price   IS NULL OR p.price <= p_max_price)
    ORDER BY p.rating DESC, p.price ASC;
END$$
DELIMITER ;

-- Search Electronics under 30000
CALL SearchProducts(NULL, 1, NULL, 30000.00);
```

▶ **Output**

```
+------------+--------------+-------------+----------+-----------+--------+
| product_id | product_name | category    | price    | stock_qty | rating |
+------------+--------------+-------------+----------+-----------+--------+
|          1 | Smartphone X | Electronics | 25000.00 |        50 |   4.50 |
+------------+--------------+-------------+----------+-----------+--------+
```

Procedure S3: GenerateSalesReport — Category-wise Revenue Report

Theory: This reporting procedure generates a sales summary grouped by product category, showing total units sold, gross revenue, and average order value. An optional date range IN parameter filters the report period.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE GenerateSalesReport(
    IN p_from_date DATE,
    IN p_to_date   DATE
)
BEGIN
    SELECT c.category_name,
           COUNT(DISTINCT o.order_id)        AS total_orders,
           SUM(oi.quantity)                  AS units_sold,
           SUM(oi.quantity * oi.unit_price)  AS gross_revenue,
           ROUND(AVG(o.total_amount),2)       AS avg_order_value,
           MAX(oi.unit_price)                AS highest_ticket
    FROM   categories c
    JOIN   products p   ON c.category_id = p.category_id
    JOIN   order_items oi ON p.product_id = oi.product_id
    JOIN   orders o       ON oi.order_id  = o.order_id
    WHERE  o.status != 'Cancelled'
      AND  (p_from_date IS NULL OR DATE(o.order_date) >= p_from_date)
      AND  (p_to_date   IS NULL OR DATE(o.order_date) <= p_to_date)
    GROUP BY c.category_name
    ORDER BY gross_revenue DESC;
END$$
DELIMITER ;

-- Full report (no date filter)
CALL GenerateSalesReport(NULL, NULL);
```

▶ Output

```
+-------------+--------------+------------+---------------+-----------------+------------
----+
| category    | total_orders | units_sold | gross_revenue | avg_order_value |
highest_ticket |
+-------------+--------------+------------+---------------+-----------------+------------
----+
| Electronics |            2 |          2 |    100000.00  |        50299.50 |
75000.00  |
| Clothing    |            2 |          3 |      1797.00  |        13524.00 |
599.00  |
| Books       |            1 |          1 |       850.00  |          850.00 |
850.00  |
+-------------+--------------+------------+---------------+-----------------+------------
----+
```

Procedure S4: UpdateProductStock — Bulk Stock Adjustment

Theory: An administrative procedure that accepts a product ID, adjustment type (ADD or DEDUCT), and quantity. It validates the operation and updates stock, demonstrating INOUT parameters and conditional DML with error signalling.

▶ Code

```
DELIMITER $$
CREATE PROCEDURE UpdateProductStock(
    IN    p_product_id  INT,
    IN    p_action      VARCHAR(10),   -- 'ADD' or 'DEDUCT'
    IN    p_qty         INT,
    OUT   p_new_stock   INT,
    OUT   p_message     VARCHAR(200)
)
BEGIN
```

```
    DECLARE v_current INT;

    SELECT stock_qty INTO v_current
    FROM products WHERE product_id = p_product_id;

    IF v_current IS NULL THEN
        SET p_message   = 'Error: Product not found.';
        SET p_new_stock = -1;
    ELSEIF p_action = 'DEDUCT' AND v_current < p_qty THEN
        SET p_message   = CONCAT('Error: Only ', v_current, ' units available.');
        SET p_new_stock = v_current;
    ELSEIF p_action = 'ADD' THEN
        UPDATE products SET stock_qty = stock_qty + p_qty
        WHERE product_id = p_product_id;
        SET p_new_stock = v_current + p_qty;
        SET p_message   = CONCAT('Stock added. New stock: ', p_new_stock);
    ELSEIF p_action = 'DEDUCT' THEN
        UPDATE products SET stock_qty = stock_qty - p_qty
        WHERE product_id = p_product_id;
        SET p_new_stock = v_current - p_qty;
        SET p_message   = CONCAT('Stock deducted. New stock: ', p_new_stock);
    ELSE
        SET p_message   = 'Error: Invalid action. Use ADD or DEDUCT.';
        SET p_new_stock = v_current;
    END IF;
END$$
DELIMITER ;

CALL UpdateProductStock(1, 'ADD', 100, @ns, @msg);
SELECT @ns AS new_stock, @msg AS message;
```

▶ Output

```
+-----------+-------------------------------+
| new_stock | message                       |
+-----------+-------------------------------+
|       150 | Stock added. New stock: 150   |
+-----------+-------------------------------+
```

STORED FUNCTIONS — shopsphere

Function S1: GetCartTotal — Total Value of a User's Cart

Theory: Returns the total monetary value of all items currently in a user's cart. This function is directly embeddable in SELECT statements, making it easy to display cart totals without a complex subquery each time.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION GetCartTotal(p_user_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;
    SELECT IFNULL(SUM(ci.quantity * p.price), 0.00) INTO v_total
    FROM   cart_items ci
    JOIN   products p ON ci.product_id = p.product_id
    WHERE  ci.user_id = p_user_id;
    RETURN v_total;
END$$
DELIMITER ;

SELECT u.user_id, u.full_name,
       GetCartTotal(u.user_id) AS cart_total
FROM   users u
ORDER BY cart_total DESC;
```

▶ **Output**

```
+---------+------------+------------+
| user_id | full_name  | cart_total |
+---------+------------+------------+
|       3 | Carol Nair |  75000.00  |
|       2 | Bob Das    |  25000.00  |
|       1 | Alice Roy  |    850.00  |
+---------+------------+------------+
```

Function S2: GetDiscountedPrice — Apply Category Discount

Theory: This function applies a percentage discount to a product's price based on its category ID and returns the discounted price. It demonstrates conditional logic inside a function reusable in pricing SELECT queries.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION GetDiscountedPrice(
    p_product_id   INT,
    p_discount_pct DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_price    DECIMAL(10,2);
    DECLARE v_discount DECIMAL(10,2);

    SELECT price INTO v_price
    FROM products WHERE product_id = p_product_id;

    IF v_price IS NULL THEN RETURN 0.00; END IF;

    SET v_discount = v_price * (p_discount_pct / 100);
    RETURN ROUND(v_price - v_discount, 2);
END$$
DELIMITER ;

-- Show all products with 10% discount applied
```

```
SELECT product_name,
       price                                AS original_price,
       GetDiscountedPrice(product_id, 10)   AS price_after_10pct_off,
       GetDiscountedPrice(product_id, 20)   AS price_after_20pct_off
FROM   products;
```

▶ **Output**

```
+------------------+----------------+----------------------+----------------------+
| product_name     | original_price | price_after_10pct_off| price_after_20pct_off|
+------------------+----------------+----------------------+----------------------+
| Smartphone X     |       25000.00 |             22500.00 |             20000.00 |
| Laptop Pro       |       75000.00 |             67500.00 |             60000.00 |
| Cotton T-Shirt   |         599.00 |               539.10 |               479.20 |
| DBMS Textbook    |         850.00 |               765.00 |               680.00 |
+------------------+----------------+----------------------+----------------------+
```

Function S3: GetCustomerLifetimeValue — Total Spend per User

Theory: Returns the cumulative spend of a customer across all non-cancelled orders. This CLV metric is used inline in SELECT to segment customers without writing a correlated subquery each time.

▶ **Code**

```
DELIMITER $$
CREATE FUNCTION GetCustomerLifetimeValue(p_user_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_clv DECIMAL(12,2) DEFAULT 0.00;
    SELECT IFNULL(SUM(total_amount), 0.00) INTO v_clv
    FROM   orders
    WHERE  user_id = p_user_id
      AND  status NOT IN ('Cancelled');
    RETURN v_clv;
END$$
DELIMITER ;

SELECT u.full_name,
       GetCustomerLifetimeValue(u.user_id) AS lifetime_value,
       CASE
           WHEN GetCustomerLifetimeValue(u.user_id) >= 50000 THEN 'Platinum'
           WHEN GetCustomerLifetimeValue(u.user_id) >= 20000 THEN 'Gold'
           WHEN GetCustomerLifetimeValue(u.user_id) > 0      THEN 'Silver'
           ELSE 'New'
       END AS loyalty_tier
FROM users u
ORDER BY lifetime_value DESC;
```

▶ **Output**

```
+------------+----------------+--------------+
| full_name  | lifetime_value | loyalty_tier |
+------------+----------------+--------------+
| Bob Das    |       75000.00 | Platinum     |
| Alice Roy  |       25599.00 | Gold         |
| Carol Nair |        1198.00 | Silver       |
+------------+----------------+--------------+
```

Function S4: GetRatingLabel — Convert Numeric Rating to Label

Theory: Converts a numeric product rating into a descriptive label (Excellent, Good, Average, Poor). This function is used inline in product listing queries to display user-friendly labels without repeating CASE expressions.

▶ Code

```
DELIMITER $$
CREATE FUNCTION GetRatingLabel(p_rating DECIMAL(3,2))
RETURNS VARCHAR(20)
DETERMINISTIC NO SQL
BEGIN
    DECLARE v_label VARCHAR(20);
    IF     p_rating >= 4.5 THEN SET v_label = 'Excellent';
    ELSEIF p_rating >= 3.5 THEN SET v_label = 'Good';
    ELSEIF p_rating >= 2.5 THEN SET v_label = 'Average';
    ELSEIF p_rating >= 1.0 THEN SET v_label = 'Poor';
    ELSE                        SET v_label = 'Not Rated';
    END IF;
    RETURN v_label;
END$$
DELIMITER ;

SELECT product_name, rating,
       GetRatingLabel(rating) AS rating_label
FROM   products
ORDER BY rating DESC;
```

▶ Output

```
+------------------+--------+--------------+
| product_name     | rating | rating_label |
+------------------+--------+--------------+
| DBMS Textbook    |   4.80 | Excellent    |
| Laptop Pro       |   4.70 | Excellent    |
| Smartphone X     |   4.50 | Excellent    |
| Cotton T-Shirt   |   4.20 | Good         |
+------------------+--------+--------------+
```

Function S5: GetStockStatus — Stock Availability Label

Theory: Returns a text label for a product's stock level: Out of Stock, Critical, Low Stock, or In Stock. Embeddable directly in catalogue SELECT queries to display availability without application-level conditionals.

▶ Code

```
DELIMITER $$
CREATE FUNCTION GetStockStatus(p_product_id INT)
RETURNS VARCHAR(20)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_stock INT DEFAULT 0;
    SELECT stock_qty INTO v_stock
    FROM products WHERE product_id = p_product_id;

    IF     v_stock = 0  THEN RETURN 'Out of Stock';
    ELSEIF v_stock <= 5 THEN RETURN 'Critical';
    ELSEIF v_stock <= 20 THEN RETURN 'Low Stock';
    ELSE                      RETURN 'In Stock';
    END IF;
END$$
DELIMITER ;

SELECT product_name, stock_qty,
       GetStockStatus(product_id) AS stock_status
FROM   products;
```

▶ Output

```
+------------------+-----------+--------------+
| product_name     | stock_qty | stock_status |
+------------------+-----------+--------------+
| Smartphone X     |       150 | In Stock     |
| Laptop Pro       |        20 | Low Stock    |
| Cotton T-Shirt   |       200 | In Stock     |
| DBMS Textbook    |       100 | In Stock     |
+------------------+-----------+--------------+
```

Summary of All Procedures and Functions

The table below lists all 18 stored routines implemented across both databases:

#	Name	Type	Database	Purpose
A1	**MakeBooking**	Procedure	airlinedb	Create booking with validation & OUT param
A2	**CancelBooking**	Procedure	airlinedb	Cancel booking; conditional refund; INOUT
A3	**FlightRevenueReport**	Procedure	airlinedb	Aggregate revenue report by flight/airline
A4	**GetPassengerHistory**	Procedure	airlinedb	Full travel history for a passenger
A5	**GetAvailableSeats**	Function	airlinedb	Returns available seat count for a flight
A6	**CalculateTicketPrice**	Function	airlinedb	Dynamic pricing by class (Economy/Biz/First)
A7	**GetPassengerBookingCount**	Function	airlinedb	Total confirmed bookings for a passenger
A8	**GetFlightLoadFactor**	Function	airlinedb	Occupancy % — usable in WHERE / SELECT
S1	**PlaceOrder**	Procedure	shopsphere	Cart-to-order checkout with cursor & stock
S2	**SearchProducts**	Procedure	shopsphere	Flexible filtered product search
S3	**GenerateSalesReport**	Procedure	shopsphere	Category-wise revenue report with date range
S4	**UpdateProductStock**	Procedure	shopsphere	ADD / DEDUCT stock with validation
S5	**GetCartTotal**	Function	shopsphere	Returns current cart value for a user
S6	**GetDiscountedPrice**	Function	shopsphere	Applies discount % to product price
S7	**GetCustomerLifetimeValue**	Function	shopsphere	Cumulative non-cancelled order spend
S8	**GetRatingLabel**	Function	shopsphere	Converts numeric rating to text label
S9	**GetStockStatus**	Function	shopsphere	Returns stock availability label

Conclusion

For implementing business logic in MySQL, stored procedures and functions are crucial. Stored procedures are good choices for doing multistep logic, whereas functions should be used for doing calculations within queries in simple cases.

For the assignment, I implemented several procedures and functions in two different databases – AirlineDB and shopsphere. For these stored routines, I used different kinds of arguments including input, output, and both together. Variables were used for making calculations, and different kinds of logical operators for making decisions.

Moreover, I learned about the use of cursors in procedures, which allowed me to work with records one by one. Error handling was done to avoid problems related to data integrity. Functions were useful to make reusable calculations inside queries such as SELECT.

This assignment allowed me to get a better understanding of how to organize logic in MySQL efficiently. Procedures are suitable for multistep logic and functions are used for simple calculations only.